



南開大學  
Nankai University

计算机学院  
语音信息处理实验报告

实验三：使用 **Facodec** 实现对语音离散  
化、特征解耦、重构以及音色转换

姓名：王一诺  
学号：2312331  
专业：计算机科学与技术

2026 年 6 月 5 日

## 目录

|          |                             |           |
|----------|-----------------------------|-----------|
| <b>1</b> | <b>实验目标与实验内容</b>            | <b>2</b>  |
| <b>2</b> | <b>实验环境配置与运行结果</b>          | <b>2</b>  |
| 2.1      | 环境配置 . . . . .              | 2         |
| 2.2      | 运行方式 . . . . .              | 2         |
| <b>3</b> | <b>实验原理</b>                 | <b>3</b>  |
| 3.1      | FACodec 语音离散化原理 . . . . .   | 3         |
| 3.2      | FACodec 语音特征解耦原理 . . . . .  | 4         |
| <b>4</b> | <b>代码实现与音色转换</b>            | <b>5</b>  |
| 4.1      | 基础重构流程 . . . . .            | 5         |
| 4.2      | 音色转换实现 . . . . .            | 5         |
| <b>5</b> | <b>Shape 输出及张量维度解释</b>      | <b>8</b>  |
| <b>6</b> | <b>使用实验一工具绘制波形图与频谱图</b>     | <b>9</b>  |
| 6.1      | 绘图方法 . . . . .              | 9         |
| 6.2      | 原始音频与重构音频对比 . . . . .       | 10        |
| <b>7</b> | <b>音色转换结果与对比分析</b>          | <b>10</b> |
| 7.1      | speaker1 转换为自己的音色 . . . . . | 10        |
| 7.2      | speaker2 转换为自己的音色 . . . . . | 11        |
| <b>8</b> | <b>实验总结</b>                 | <b>12</b> |

## 1 实验目标与实验内容

本实验使用 FACodec 模型完成语音离散化、语音特征解耦、语音重构和音色转换。FACodec 由编码器 FACodecEncoder 和解码器 FACodecDecoder 两部分组成：编码器将原始语音波形压缩为连续潜在表示，解码器进一步对潜在表示进行向量量化与特征解耦，并根据说话人音色嵌入重构语音。

本次实验主要完成以下内容：

1. 配置 conda 环境并运行实验代码，确保程序能够成功输出 **Successfully Reconstruct!**。
2. 使用 FACodec 对语音进行离散化，将连续语音特征转换为离散码本索引。
3. 使用 FACodec 将语音特征解耦为 Prosody、Content、Acoustic Detail 和 Timbre/Speaker。
4. 使用解耦后的特征重构语音，生成 **speaker2\_rec.wav**。
5. 录制自己的语音 **my\_voice.wav**，提取其中的 Timbre/Speaker 音色信息；提取 **samples/speaker1.wav** 的 Prosody、Content 和 Acoustic Detail，重构为自己的音色。
6. 使用实验一中的绘图工具打印原始音频、重构音频、音色转换前后音频的波形图和频谱图，并进行对比分析。

## 2 实验环境配置与运行结果

### 2.1 环境配置

根据实验指导手册，本实验使用 conda 创建独立环境，并安装 FACodec 运行所需依赖。主要环境配置命令如下：

```
1 conda create -n facodec python=3.10
2 conda activate facodec
3 pip3 install torch==2.1.2 torchaudio==2.1.2
4 pip install librosa==0.10.1
5 pip install pyworld
6 pip install einops
7 pip install soundfile
8 pip install "numpy<2"
9
```

实际运行中，librosa 会导入 pkg\_resources。若出现 `ModuleNotFoundError: No module named 'pkg_resources'`，需要安装或降级 setuptools。本实验最终环境能够正常运行代码。

### 2.2 运行方式

在 FACodec 代码目录下运行：

```
1 python test.py
2
```

程序运行后完成三类音频生成：

1. 重构音频: `samples/speaker2_rec.wav`
2. speaker1 音色转换音频: `samples/speaker1_rec_with_my_voice_spk.wav`
3. speaker2 音色转换音频: `samples/speaker2_rec_with_my_voice_spk.wav`

实验运行结果截图如图 2.1 所示。程序最终打印了 `Successfully Convert Timbre!`、`Successfully Convert Speaker2 Timbre!` 和 `Successfully Reconstruct!`，说明重构和音色转换流程均已完成。

```
(fcodec) D:\语音信息处理\实验3\Facodec>python test.py
D:\anaconda3\envs\facodec\lib\site-packages\librosa\core\intervals.py:8: UserWarning: pkg_resources is deprecated as
he pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setu
from pkg_resources import resource_filename
D:\anaconda3\envs\facodec\lib\site-packages\torch\nn\utils\weight_norm.py:30: UserWarning: torch.nn.utils.weight_norm
.
warnings.warn("torch.nn.utils.weight_norm is deprecated in favor of torch.nn.utils.parametrizations.weight_norm.")
Test Audio Shape: torch.Size([1, 1, 152418])
Encoder_out Shape: torch.Size([1, 256, 762])
Prosody Code Shape: torch.Size([1, 1, 762])
Content Code Shape: torch.Size([2, 1, 762])
Residual Code Shape: torch.Size([3, 1, 762])
Prosody Embedding Shape: torch.Size([1, 256, 762])
Content Embedding Shape: torch.Size([1, 256, 762])
Detail Embedding Shape: torch.Size([1, 256, 762])
Speaker Embedding Shape: torch.Size([1, 256])
Source Audio Shape: torch.Size([1, 1, 125677])
Timbre Audio Shape: torch.Size([1, 1, 158880])
Source Encoder_out Shape: torch.Size([1, 256, 628])
Timbre Encoder_out Shape: torch.Size([1, 256, 794])
Source Prosody Embedding Shape: torch.Size([1, 256, 628])
Source Content Embedding Shape: torch.Size([1, 256, 628])
Source Detail Embedding Shape: torch.Size([1, 256, 628])
New Speaker Embedding Shape: torch.Size([1, 256])
Residual Scale For Timbre Conversion: 0.0
Timbre Converted Audio Shape: torch.Size([1, 1, 125600])
Successfully Convert Timbre!
Speaker2 Prosody Embedding Shape: torch.Size([1, 256, 762])
Speaker2 Content Embedding Shape: torch.Size([1, 256, 762])
Speaker2 Detail Embedding Shape: torch.Size([1, 256, 762])
Speaker2 Timbre Converted Audio Shape: torch.Size([1, 1, 152400])
Successfully Convert Speaker2 Timbre!
Reconstruct Audio Shape: torch.Size([1, 1, 152400])
Successfully Reconstruct!
```

图 2.1: FACodec 实验程序运行结果截图

运行日志中的两个 warning 分别来自 librosa 对 `pkg_resources` 的弃用提示，以及 PyTorch 中 `weight_norm` 接口的弃用提示。这两个 warning 不影响实验代码运行，也不影响输出音频文件生成。

## 3 实验原理

### 3.1 FACodec 语音离散化原理

原始语音是连续时间波形，在计算机中表现为一维采样序列。直接处理原始波形会包含大量冗余信息，例如短时幅度波动、背景噪声以及与语义无关的细节。FACodec 首先使用编码器将原始波形映射到低帧率的连续潜在空间。例如实验中 `speaker2.wav` 的输入形状为：

```
torch.Size([1, 1, 152418])
```

经过编码器后得到：

```
torch.Size([1, 256, 762])
```

其中三个维度分别表示：

`[batch_size, dimension, time_length]`

这里 `batch_size=1` 表示一次处理一个音频样本，`dimension=256` 表示每个时间帧的特征维度，`time_length=762` 表示编码器下采样后的时间帧数。

在 `FACodecDecoder` 中，向量量化模块会将每个时间帧的连续向量映射到码本中最近的中心向量，并用离散码本索引表示该时间帧的特征。离散化后的结果不再是任意连续值，而是有限码本中的编号。例如 Prosody 离散码形状为：

`torch.Size([1, 1, 762])`

表示 1 个 Prosody 量化器、1 个 batch、762 个时间帧。

语音离散化的作用主要包括：

1. **压缩语音表示。** 语音从高采样率波形变为较短的时间帧序列，降低后续建模成本。
2. **降低连续扰动。** 离散码本约束可以减少原始波形中无关节对语音表示的影响。
3. **便于重构与生成。** 离散表示更适合被语音生成模型、语音转换模型或语言模型进一步处理。
4. **为特征解耦提供基础。** `FACodec` 将不同量化器分配给 Prosody、Content 和 Acoustic Detail，从而实现更清晰的特征分离。

### 3.2 `FACodec` 语音特征解耦原理

`FACodec` 的核心特点是将语音中的不同因素进行解耦。实验代码中，解码器将编码器输出拆分为以下四类信息：

表 1: `FACodec` 解耦语音特征

| 特征              | 含义                            |
|-----------------|-------------------------------|
| Prosody         | 语调、节奏、韵律、音高变化等信息，决定一句话如何被说出来。 |
| Content         | 语音内容和音素相关信息，决定说了什么。           |
| Acoustic Detail | 残差声学细节，包括高频细节、音质细节和部分个体发声差异。  |
| Timbre/Speaker  | 说话人音色、声纹和发声习惯等信息，决定由谁的声音说。    |

实验代码中，离散码本索引通过如下方式划分：

```
1 prosody_code = vq_id[:1]
2 content_code = vq_id[1:3]
3 detail_code = vq_id[3:]
4
```

连续量化嵌入通过如下方式获得：

```
1 _, _, _, quantized = fa_decoder.quantize(encoder_out)
2 prosody = quantized[0]
3 content = quantized[1]
4 detail = quantized[2]
5
```

重构时，将 Prosody、Content 和 Acoustic Detail 相加，作为待解码的主体语音表示；同时传入说话人嵌入 `spk_embs` 控制音色：

```
1 all_embs = prosody + content + detail
2 rec_wav = fa_decoder.inference(all_embs, spk_embs)
3
```

因此，FACodec 的特征解耦使得语音的内容、韵律、细节和音色可以被相对独立地控制。这也是音色转换能够实现的原因：保持源音频的 Prosody、Content 和 Acoustic Detail，同时替换 Timbre/Speaker，即可得到内容不变但音色改变的语音。

## 4 代码实现与音色转换

### 4.1 基础重构流程

基础重构流程使用 `speaker2.wav` 作为输入。首先用编码器得到 `encoder_out`，再用解码器得到 Prosody、Content、Detail 和 Speaker Embedding，最后组合前三类特征并使用原说话人嵌入进行重构，生成 `speaker2_rec.wav`。

### 4.2 音色转换实现

实验指导要求：录制一段自己的语音提取 Timbre/Speaker，提取 `samples/speaker1.wav` 的 Prosody、Content 和 Acoustic Detail 进行重构，使重构后的音频保持 `speaker1.wav` 的语调和内容，但音色转换为自己的音色。

本实验中，自录语音文件命名为 `my_voice.wav`，放置在 `samples` 目录下。音色转换代码截图如图 4.2 和图 4.3 所示。

```
#### Step5: 利用Facodec的代码实现音色转换效果。
source_wav_file = "speaker1"
timbre_wav_file = "my_voice"
source_wav = load_audio(f"samples/{source_wav_file}.wav")
timbre_wav = load_audio(f"samples/{timbre_wav_file}.wav")
print("Source Audio Shape: ", source_wav.shape)
print("Timbre Audio Shape: ", timbre_wav.shape)

source_encoder_out = fa_encoder(source_wav)
timbre_encoder_out = fa_encoder(timbre_wav)
print("Source Encoder_out Shape: ", source_encoder_out.shape)
print("Timbre Encoder_out Shape: ", timbre_encoder_out.shape)

_, _, _, new_spk_embs = fa_decoder(
    timbre_encoder_out, eval_vq=False, vq=True
)

_, _, _, source_quantized = fa_decoder.quantize(source_encoder_out)
source_prosody = source_quantized[0]
print("Source Prosody Embedding Shape:", source_prosody.shape)
source_content = source_quantized[1]
print("Source Content Embedding Shape:", source_content.shape)
source_detail = source_quantized[2]
print("Source Detail Embedding Shape:", source_detail.shape)
print("New Speaker Embedding Shape:", new_spk_embs.shape)

residual_scale = 0.0
print("Residual Scale For Timbre Conversion:", residual_scale)
source_all_embs = source_prosody + source_content + residual_scale * source_detail
rec_wav_with_new_spk = fa_decoder.inference(source_all_embs, new_spk_embs)
print("Timbre Converted Audio Shape: ", rec_wav_with_new_spk.shape)
sf.write(
    f"samples/{source_wav_file}_rec_with_{timbre_wav_file}_spk.wav",
    rec_wav_with_new_spk[0][0].cpu().numpy(),
    16000,
)
print("Successfully Convert Timbre!")
```

图 4.2: 音色转换 Step5 代码截图一



```

speaker2_wav_file = "speaker2"
speaker2_wav = load_audio(f"samples/{speaker2_wav_file}.wav")
speaker2_encoder_out = fa_encoder(speaker2_wav)
_, _, _, speaker2_quantized = fa_decoder.quantize(speaker2_encoder_out)
speaker2_prosody = speaker2_quantized[0]
print("Speaker2 Prosody Embedding Shape:", speaker2_prosody.shape)
speaker2_content = speaker2_quantized[1]
print("Speaker2 Content Embedding Shape:", speaker2_content.shape)
speaker2_detail = speaker2_quantized[2]
print("Speaker2 Detail Embedding Shape:", speaker2_detail.shape)

speaker2_all_embs = speaker2_prosody + speaker2_content + residual_scale * speaker2_detail
speaker2_rec_wav_with_new_spk = fa_decoder.inference(speaker2_all_embs, new_spk_embs)
print("Speaker2 Timbre Converted Audio Shape: ", speaker2_rec_wav_with_new_spk.shape)
sf.write(
    f"samples/{speaker2_wav_file}_rec_with_{timbre_wav_file}_spk.wav",
    speaker2_rec_wav_with_new_spk[0][0].cpu().numpy(),
    16000,
)
print("Successfully Convert Speaker2 Timbre!")

```

图 4.3: 音色转换 Step5 代码截图二

核心实现逻辑如下:

```

1 source_wav_file = "speaker1"
2 timbre_wav_file = "my_voice"
3 source_wav = load_audio(f"samples/{source_wav_file}.wav")
4 timbre_wav = load_audio(f"samples/{timbre_wav_file}.wav")
5
6 source_encoder_out = fa_encoder(source_wav)
7 timbre_encoder_out = fa_encoder(timbre_wav)
8
9 _, _, _, new_spk_embs = fa_decoder(
10     timbre_encoder_out, eval_vq=False, vq=True
11 )
12 _, _, _, source_quantized = fa_decoder.quantize(source_encoder_out)
13 source_prosody = source_quantized[0]
14 source_content = source_quantized[1]
15 source_detail = source_quantized[2]
16
17 residual_scale = 0.0
18 source_all_embs = source_prosody + source_content + residual_scale * source_detail
19 rec_wav_with_new_spk = fa_decoder.inference(source_all_embs, new_spk_embs)
20

```

其中,source\_wav 对应 speaker1.wav,负责提供 Prosody、Content 和 Acoustic Detail;timbre\_wav 对应自录语音 my\_voice.wav,负责提供新的 Timbre/Speaker。最终生成的 speaker1\_rec\_with\_my\_voice\_spk.wav 保持 speaker1.wav 的语音内容和韵律,同时音色向自录语音靠近。

需要说明的是,Acoustic Detail/Residual 中会携带一部分源说话人的声学细节和音色信息。如果直接使用:



```

1 source_all_embs = source_prosody + source_content + source_detail
2

```

转换结果可能仍然非常接近 `speaker1.wav` 的原始音色。为了使音色转换效果更明显，我尝试保留 `source_detail` 的提取和 Shape 打印，但在合成时使用：

```

1 residual_scale = 0.0
2 source_all_embs = source_prosody + source_content + residual_scale * source_detail
3

```

这样可以降低源说话人音色通过 `residual` 泄漏到输出语音中的程度。若希望在音质和音色转换强度之间折中，可以尝试将 `residual_scale` 调整为 0.2 或 0.5。

除指导手册要求的 `speaker1.wav` 音色转换外，本实验还额外将 `speaker2.wav` 的音色转换为自己的音色，生成 `speaker2_rec_with_my_voice_spk.wav`，用于进一步对比。

## 5 Shape 输出及张量维度解释

本节解释代码中所有 `print(...shape)` 输出的张量维度含义。实验运行得到的 Shape 如表 2 所示。

表 2: 代码中所有 Shape 输出的维度解释

| 打印项                     | 实际输出形状         | 维度含义                                                                            |
|-------------------------|----------------|---------------------------------------------------------------------------------|
| Test Audio Shape        | [1, 1, 152418] | [batch_size, channel, sample_length]。1 个样本，单声道，152418 个采样点。                     |
| Encoder_out Shape       | [1, 256, 762]  | [batch_size, dimension, time_length]。1 个样本，每帧 256 维，下采样后 762 帧。                 |
| Prosody Code Shape      | [1, 1, 762]    | [num_prosody_quantizer, batch_size, time_length]。1 个 Prosody 量化器，1 个样本，762 帧。   |
| Cotent Code Shape       | [2, 1, 762]    | [num_content_quantizer, batch_size, time_length]。2 个 Content 量化器，1 个样本，762 帧。   |
| Residual Code Shape     | [3, 1, 762]    | [num_residual_quantizer, batch_size, time_length]。3 个 Residual 量化器，1 个样本，762 帧。 |
| Prosody Embedding Shape | [1, 256, 762]  | [batch_size, dimension, time_length]。Prosody 连续量化嵌入。                            |
| Content Embedding Shape | [1, 256, 762]  | [batch_size, dimension, time_length]。Content 连续量化嵌入。                            |
| Detail Embedding Shape  | [1, 256, 762]  | [batch_size, dimension, time_length]。Acoustic Detail 连续量化嵌入。                    |
| Speaker Embedding Shape | [1, 256]       | [batch_size, speaker_embedding_dimension]。1 个样本，256 维说话人音色嵌入。                   |
| Source Audio Shape      | [1, 1, 125677] | [batch_size, channel, sample_length]。 <code>speaker1.wav</code> 的波形输入。          |
| Timbre Audio Shape      | [1, 1, 158880] | [batch_size, channel, sample_length]。自录语音 <code>my_voice.wav</code> 的波形输入。      |

| 打印项                                   | 实际输出形状         | 维度含义                                                                        |
|---------------------------------------|----------------|-----------------------------------------------------------------------------|
| Source Encoder_out Shape              | [1, 256, 628]  | [batch_size, dimension, time_length]。<br>speaker1.wav 编码后的潜在特征。             |
| Timbre Encoder_out Shape              | [1, 256, 794]  | [batch_size, dimension, time_length]。<br>my_voice.wav 编码后的潜在特征。             |
| Source Prosody Embedding Shape        | [1, 256, 628]  | [batch_size, dimension, time_length]。<br>speaker1.wav 的 Prosody 特征。         |
| Source Content Embedding Shape        | [1, 256, 628]  | [batch_size, dimension, time_length]。<br>speaker1.wav 的 Content 特征。         |
| Source Detail Embedding Shape         | [1, 256, 628]  | [batch_size, dimension, time_length]。<br>speaker1.wav 的 Acoustic Detail 特征。 |
| New Speaker Embedding Shape           | [1, 256]       | [batch_size, speaker_embedding_dimension]。<br>从自录语音中提取的新说话人音色嵌入。            |
| Timbre Converted Audio Shape          | [1, 1, 125600] | [batch_size, channel, sample_length]。<br>speaker1 转换为自己音色后的波形。              |
| Speaker2 Prosody Embedding Shape      | [1, 256, 762]  | [batch_size, dimension, time_length]。<br>speaker2.wav 的 Prosody 特征。         |
| Speaker2 Content Embedding Shape      | [1, 256, 762]  | [batch_size, dimension, time_length]。<br>speaker2.wav 的 Content 特征。         |
| Speaker2 Detail Embedding Shape       | [1, 256, 762]  | [batch_size, dimension, time_length]。<br>speaker2.wav 的 Acoustic Detail 特征。 |
| Speaker2 Timbre Converted Audio Shape | [1, 1, 152400] | [batch_size, channel, sample_length]。<br>speaker2 转换为自己音色后的波形。              |
| Reconstruct Audio Shape               | [1, 1, 152400] | [batch_size, channel, sample_length]。<br>speaker2.wav 重构后的波形。               |

从表中可以看出，音频波形通常使用 [batch\_size, channel, sample\_length] 表示；编码器输出和连续量化嵌入通常使用 [batch\_size, dimension, time\_length] 表示；离散码本索引则将量化器数量放在第一维，即 [num\_quantizer, batch\_size, time\_length]。

## 6 使用实验一工具绘制波形图与频谱图

### 6.1 绘图方法

实验指导要求使用实验一的工具打印原始音频、重构音频的波形图和频谱图对比。实验一中使用的主要工具包括：

1. librosa.load(): 加载音频。
2. matplotlib.pyplot: 绘制图像。
3. pandas.Series(...).plot(): 绘制原始波形。
4. librosa.stft(): 计算短时傅里叶变换。
5. librosa.amplitude\_to\_db(): 将幅度谱转换为 dB 标度。
6. librosa.display.specshow(): 绘制频谱图。

此处我使用 notebook `plot_compare.ipynb` 生成对比图。每张图包含四个子图：左上为第一个音频波形，右上为第二个音频波形，左下为第一个音频频谱图，右下为第二个音频频谱图，并明确标注了比较的是哪两个音频。

## 6.2 原始音频与重构音频对比

图 6.4 展示了 `speaker2.wav` 与 `speaker2_rec.wav` 的波形图和频谱图对比。

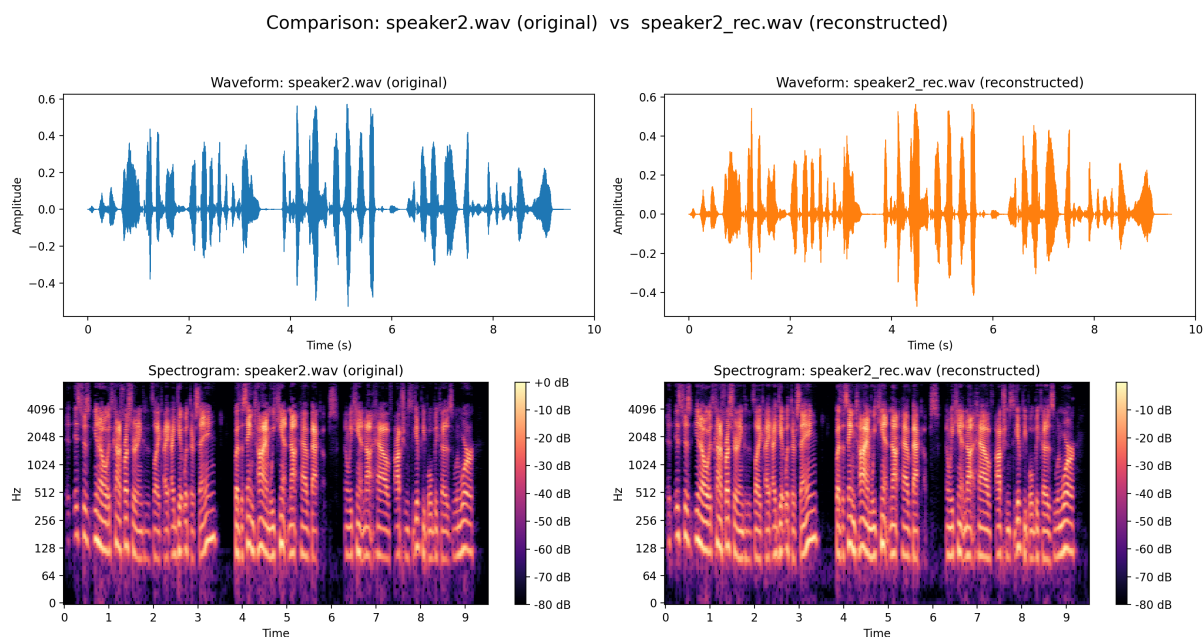


图 6.4: `speaker2.wav` 原始音频与 `speaker2_rec.wav` 重构音频的波形图和频谱图对比

从波形图看，重构音频与原始音频在整体时长、能量起伏和语音活动区间上基本一致，说明 FCo-dec 能够保留主要语音结构。从频谱图看，两者的主要能量分布区域接近，低频和中频语音能量均被较好保留，说明重构结果保持了原始语音的主要频谱特征。由于 FCo-dec 经历了编码、量化和解码过程，重构频谱中局部细节可能与原始音频存在差异，这是有损压缩与神经声码器重建带来的正常现象。

## 7 音色转换结果与对比分析

### 7.1 speaker1 转换为自己的音色

图 7.5 展示了 `speaker1.wav` 与 `speaker1_rec_with_my_voice_spk.wav` 的波形图和频谱图对比。该转换结果使用 `speaker1.wav` 提供 Prosody、Content 和 Acoustic Detail，使用 `my_voice.wav` 提供 Timbre/Speaker。

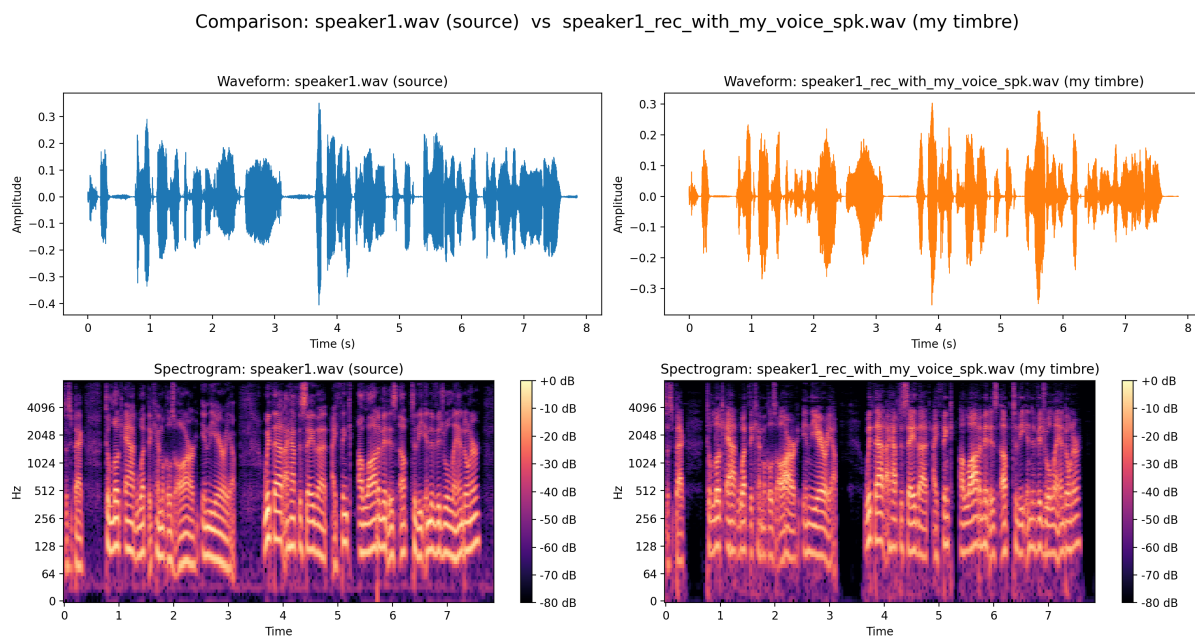


图 7.5: speaker1.wav 与转换后 speaker1\_rec\_with\_my\_voice\_spk.wav 的波形图和频谱图对比

从波形上看，转换后音频与 speaker1.wav 的时长和语音节奏保持一致，说明 Content 和 Prosody 信息被保留。从频谱上看，转换后音频的能量分布与原始 speaker1.wav 不完全相同，体现出说话人音色嵌入替换后对频谱包络的影响。听觉上，转换音频应保持 speaker1.wav 的语音内容和韵律，但音色向自录语音 my\_voice.wav 靠近。

## 7.2 speaker2 转换为自己的音色

除实验指导要求的 speaker1.wav 转换外，本实验还将 speaker2.wav 转换为自己的音色，结果如图 7.6 所示。

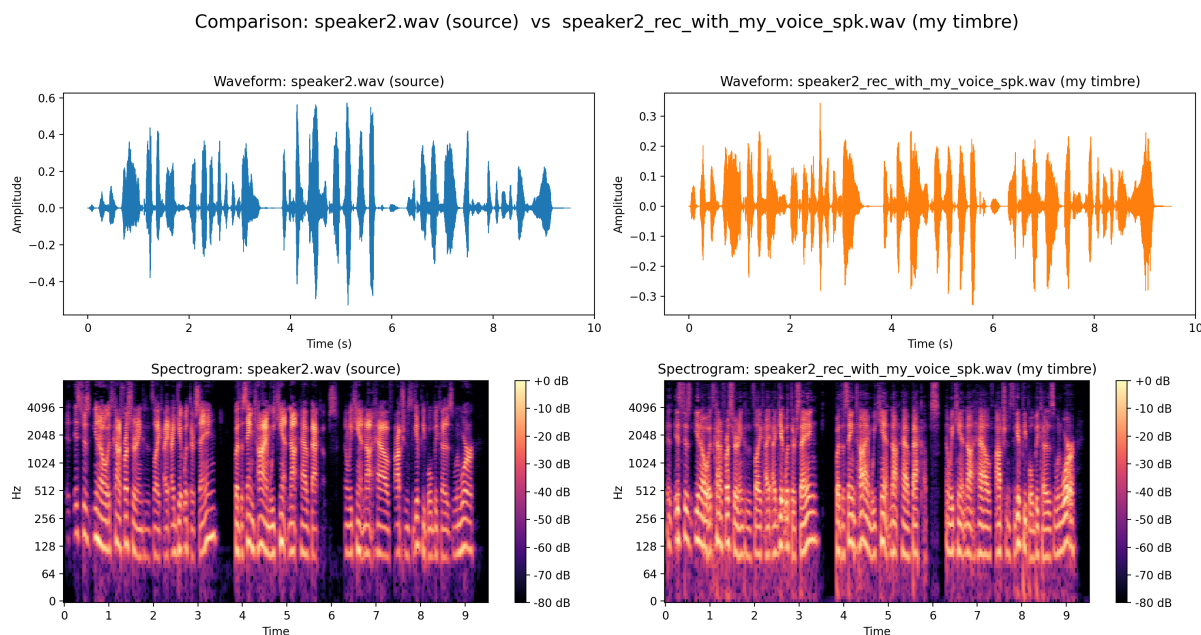


图 7.6: speaker2.wav 与转换后 speaker2\_rec\_with\_my\_voice\_spk.wav 的波形图和频谱图对比

该对比进一步说明,FACodec 的音色转换并不依赖固定源说话人。只要从目标音频中提取 Prosody、Content 和 Acoustic Detail, 并使用自录语音的 Timbre/Speaker 作为说话人条件, 就可以把不同源音频转换为自己的音色。

## 8 实验总结

本实验完成了 FFACodec 的语音离散化、特征解耦、重构和音色转换。实验结果表明:

1. FFACodecEncoder 能够将原始波形压缩为形如 `[batch_size, dimension, time_length]` 的连续潜在表示。
2. FFACodecDecoder 能够将连续潜在表示离散化, 并拆分为 Prosody、Content、Acoustic Detail 和 Timbre/Speaker 等因素。
3. 使用 `prosody + content + detail` 与原说话人嵌入可以重构原始语音。
4. 使用源音频的 Prosody、Content、Acoustic Detail 与自录语音的 Timbre/Speaker 可以实现音色转换, 使输出音频保持源语音内容和韵律, 同时音色向自录语音靠近。
5. 波形图和频谱图对比显示, 重构音频能够保持原始音频的主要语音结构和频谱能量分布; 音色转换结果在保持时长和语音节奏的同时, 频谱结构发生变化, 体现了说话人音色条件的影响。